
LLM Wiki와 AWS

지식이 쌓이는 AI를 만드는 클라우드 아키텍처

RAG의 한계를 넘어, 지식 유지보수를 자동화하는 운영 패턴

이 글이 다루는 것

- LLM Wiki가 무엇이고 기존 RAG와 어떻게 다른가
- 일반인이 이해할 수 있는 비유와 실생활 활용 예시
- AWS 서비스로 LLM Wiki를 어떻게 구성할 수 있나
- 개인, 팀, 기업 관점에서 본 비즈니스 가치

1. 어제 물어봤는데, 또 처음부터 찾고 있네

ChatGPT나 Claude 같은 AI에 자료를 올려놓고 질문해 본 적 있나요? 처음에는 신기합니다. PDF 몇 개 올려두면 척척 답을 만들어주니까요.

그런데 며칠 쓰다 보면 묘하게 답답한 순간이 옵니다. 어제 다섯 개 자료를 종합해서 좋은 답을 받았는데, 오늘 비슷한 걸 물어보면 AI가 또 처음부터 자료를 뒤지고 있어요. 어제의 결론이 어디에도 남아 있지 않거든요.

여행 준비를 떠올리면 쉽습니다. 항공권, 숙소, 맛집, 아이랑 갈 만한 곳 자료를 잔뜩 모아놓고 “우리 가족 일정에 맞는 숙소가 뭐였지?”라고 물으면, AI는 매번 처음 자료를 보는 사람처럼 답합니다. 어제 비교해서 정리해 준 결과는 어딘가 채팅방 깊은 곳에 묻혀 사라졌고요.

2. 지금까지 방식: 매번 처음부터 다시

지금 대부분의 AI 문서 활용은 **RAG(Retrieval-Augmented Generation)**라고 부르는 방식입니다. 어렵게 들리지만 원리는 간단해요. **질문이 들어올 때마다 자료에서 관련된 부분을 찾아 AI에게 보여주고 답을 만들게 하는 방식**입니다.

도서관에 비유하면 이런 거예요. 질문할 때마다 사서가 서가에서 책을 몇 권 뽑아 와 펼쳐서 답을 정리해 줍니다. 답을 듣고 나면 사서는 책을 다시 꽂고, 메모도 남기지 않습니다. 다음에 비슷한 질문을 하면? 똑같이 처음부터 책을 다시 뽑죠.

이래서는 아무리 많이 물어봐도 지식이 쌓이지 않습니다. 매번 단리 이자를 받는 셈이죠.

3. 새로운 방식: AI가 직접 위키를 쓴다

OpenAI 공동 창립자이자 테슬라 AI 총괄을 지낸 안드레이 카파시(Andrej Karpathy)가 제안한 **LLM Wiki**는 이 문제를 정면으로 뒤집습니다.

핵심 아이디어

질문할 때마다 자료를 다시 조립하지 말고, AI가 미리 정리한 위키를 만들어 계속 업데이트하게 하자.

다시 도서관 비유로 돌아가 볼까요. 이번에는 사서가 책을 뽑을 때마다 옆에 있는 노트에 핵심을 적어둡니다. 새 책이 들어오면 기존 노트를 펼쳐서 갱신하고, 모순되는 내용은 표시하고, 관련 항목끼리 연결해 둡니다. 시간이 지날수록 이 노트는 도서관 전체를 빠르게 안내하는 지도가 됩니다.

LLM Wiki도 똑같은 발상이에요. AI가 자료를 받을 때마다 마크다운 파일에 정리본을 쌓고, 페이지끼리 링크를 걸고, 새 자료가 오면 기존 페이지를 고칩니다. 사람은 그걸 읽기만 하면 됩니다.

3.1 세 층 구조: 원본, 위키, 스키마

LLM Wiki의 구조는 단순합니다. 세 층으로 나뉘어요.

- **원본 자료(Raw Sources)** – 논문, 기사, 회의록, 메모처럼 큐레이션한 소스. 절대 수정하지 않는 진실의 원천입니다.
- **위키(Wiki)** – LLM이 직접 쓰고 유지하는 마크다운 파일 모음. 요약 페이지, 개념 페이지, 비교 분석 등이 들어갑니다.
- **스키마(Schema)** – LLM이 위키를 어떤 규칙으로 운영할지 정의한 문서. CLAUDE.md나 AGENTS.md 같은 설정 파일입니다.

3.2 세 가지 핵심 작업

- **Ingest(통합)** – 새 자료를 넣으면 LLM이 읽고 요약하고 관련 페이지를 갱신합니다. 하나의 자료가 10~15개 페이지를 동시에 건드릴 수 있습니다.

- **Query(질의)** – 위키 기반으로 질문에 답합니다. 좋은 답변은 다시 위키에 새 페이지로 저장됩니다. 탐색 자체가 지식이 됩니다.
- **Lint(점검)** – 페이지 간 모순, 고아 페이지, 빠진 교차 참조를 주기적으로 점검합니다.

4. 왜 사람이 직접 못 했을까

위키 자체는 새로운 발명이 아닙니다. 위키피디아도, 회사 사내 위키도 다 같은 구조예요. 그런데 개인이나 작은 팀에서 위키를 시작하면 거의 다 실패합니다.

이유는 단순합니다. **유지보수가 너무 귀찮거든요.**

새 자료가 오면 관련 페이지 5~10개를 함께 손봐야 하고, 옛 정보가 틀린 게 되면 바꿔야 하고, 끊긴 링크를 이어야 하고. 처음엔 열심히 하지만 한두 달이면 손을 놓게 됩니다. 그러면 위키는 절반만 최신인 어정쩡한 상태로 방치되죠.

LLM은 이 부분에서 사람과 정반대입니다.

- 지루함을 안 느낍니다
- 교차 참조를 갱신하는 걸 까먹지 않습니다
- 한 번에 15개 파일을 동시에 손볼 수 있습니다

유지보수 비용이 사람보다 훨씬 낮아지니, 위키가 커질수록 가치가 올라갑니다. 사람이 위키를 포기했던 가장 큰 이유가 약해지는 거예요. 다만 “거의 0”은 아닙니다. LLM 호출 비용, 큐레이션 시간, 잘못된 정리를 검토하는 시간이 여전히 듭니다. 자세한 한계는 뒤에서 다시 다룹니다.

5. 무엇이 얼마나 좋아지나

가장 단순한 형태의 RAG(Naive RAG)와 비교하면 차이는 분명합니다.

구분	Naive RAG	LLM Wiki
지식 축적	매번 처음부터	계속 쌓임
같은 질문 반복 시	매번 새로 답 만들	정리된 결론에서 답
모순된 정보	발견 못 함	AI가 표시해 줌
사람의 부담	자료, 질문, 검토 모두	큐레이션과 검토만
새 자료 통합	단순 색인	관련 페이지 자동 갱신

카파시 본인이 약 100건의 자료, 수백 페이지 규모의 위키를 운영하면서 “복잡한 검색 시스템 없이 마크다운 파일과 인덱스 하나만으로도 충분히 작동했다”고 보고했습니다. 다만 이는 1인 개인 실험(n=1)의 사례 보고이며, 조직 환경에서의 일반화는 별도 검증이 필요합니다.

다만 RAG에는 다양한 변형이 있고, 각자 잘하는 영역이 있습니다. 다음 절에서 좀 더 균형 잡힌 시각으로 들여다봅니다.

6. 누가 어떻게 쓰는가: 5가지 대표 활용 사례

추상적인 비교보다 구체적인 사례가 이해를 돕는다. 다음 다섯 가지는 LLM Wiki가 실질적 가치를 만드는 대표 시나리오이다. 각 사례는 **도입 전 상황**, **도입 후 변화**, **핵심 가치**로 구성된다.

6.1 사례 1. 박사 과정 리서치 위키

대상: 특정 분야의 박사 과정 학생, 독립 연구자, 산업 분석가

도입 전: 100편 이상의 논문, 학회 발표, 블로그를 읽었지만, 한 달 후에는 “어떤 논문에서 무엇을 봤는지” 기억나지 않는다. 같은 논문을 두 번 읽거나, 핵심 인용을 찾으려 자료를 처음부터 다시 뒤진다.

도입 후: 새 논문을 읽을 때마다 LLM이 위키에 요약 페이지를 만들고, 기존 페이지(저자, 방법론, 결과)와 자동으로 연결한다. “A 방법론과 B 방법론의 결과 차이는?” 같은 종합 질문에 위키가 즉답한다.

핵심 가치: 100편 단위의 자료를 머릿속이 아닌 “읽을 수 있는 외부 종합 문서”로 보관. 논문 제출 시점에 인용과 비교 분석을 즉시 회수할 수 있다.

6.2 사례 2. 스타트업 기술 의사결정 위키

대상: CTO, 시니어 엔지니어, 아키텍처 결정에 관여하는 PM

도입 전: “왜 우리는 PostgreSQL 대신 DynamoDB를 골랐지?” “작년에 이 사람을 채용 거절한 이유는?” 같은 질문을 하면, 슬랙과 메일과 회의록을 뒤져야 한다. 질문한 사람은 답을 듣지 못하거나, 답을 안다고 생각한 시니어가 잘못 기억한 답을 준다.

도입 후: ADR(아키텍처 결정 기록), RFC, PR 본문, 회의록이 모두 위키 자료로 들어가고, LLM이 “decisions/” 디렉토리에 “왜 이 결정을 했는가” 페이지를 자동 생성한다. 새로운 의사결정이 생기면 기존 결정과의 일관성을 자동으로 점검한다.

핵심 가치: 조직의 의사결정 근거가 사람의 기억에서 외부 검증 가능한 문서로 이동한다. 신규 입사자가 “왜 이렇게 만들어졌는가”에 대한 답을 시니어 시간 없이 얻는다.

6.3 사례 3. 컨설팅 펌의 클라이언트별 지식 베이스

대상: 매니지먼트 컨설팅 펌, 로펌, IT 컨설팅 팀

도입 전: 클라이언트 A와의 회의록, 분석 자료, 산업 조사가 PowerPoint 100개로 흩어져 있다. 새 컨설턴트가 클라이언트 A 프로젝트에 투입되면 “무엇을 이미 분석했는가” 파악하는 데만 일주일이 걸린다.

도입 후: 클라이언트별로 분리된 위키가 자동 유지된다. “클라이언트 A에 대해 우리가 지금까지 발견한 핵심 인사이트 5가지는?” 같은 질문에 위키가 답한다. IAM으로 클라이언트별 권한이 엄격히 분리된다.

핵심 가치: 컨설턴트 회전율이 높은 환경에서 “조직의 기억”을 유지한다. 동시에 클라이언트 격리 (legal segregation)를 인프라 레벨에서 강제한다.

6.4 사례 4. 엔지니어링 팀의 장애 회고 위키

대상: SRE, 플랫폼 엔지니어링, 대규모 운영 팀

도입 전: 장애 회고 문서를 작성한 뒤 사실상 사장된다. 6개월 후 비슷한 장애가 일어나도 “비슷한 일이 있었던가” 떠올리지 못한다. 같은 실수가 반복된다.

도입 후: 회고 문서가 위키로 들어가면 LLM이 “유사 장애 패턴” 페이지를 자동 갱신한다. 새 장애가 발생하면 위키가 “2024년 3월의 X 장애와 패턴이 80% 일치합니다” 같은 알림을 제공한다.

핵심 가치: 장애의 “재발 비용”을 직접 줄인다. 한 번의 회고가 다음 장애를 막는 자산으로 누적된다.

6.5 사례 5. 의료/법률 사례 데이터베이스

대상: 임상 연구팀, 변호사 사무소, 규제 전문가

도입 전: 수백 건의 임상 사례 또는 판례가 PDF로 존재한다. “이 환자 증상과 유사한 과거 사례 5건”을 찾으려면 전문가 한 명이 반나절을 쓴다. 누락이 자주 발생한다.

도입 후: 사례별 요약 페이지가 위키에 정리되고, 핵심 변수(진단명, 증상, 처치 결과)별로 색인이 자동 갱신된다. 다만 이 영역은 5절에서 강조한 *hallucination* 누적 위험이 가장 큰 곳이며, *Trust Calibration* 워크플로와 도메인 전문가 검토가 필수다.

핵심 가치: 검색 시간이 반나절에서 분 단위로 단축된다. 그러나 위키는 의사결정 “보조 도구”이며, 최종 판단은 반드시 면허를 가진 전문가가 내려야 한다.

6.6 사례 비교 요약

사례	자료 규모	핵심 위험	성공 핵심 요소
박사 리서치	100~300건	잘못된 종합 인용	출처 fingerprint 보존
스타트업 결정	수십~수백 건	의사결정 왜곡	PR 검토 게이트
컨설팅 펌	클라이언트별 분리	정보 누출	IAM 권한 분리
장애 회고	회고 누적	패턴 오인식	자동 평가 메트릭
의료/법률	사례 수백~수천	Hallucination 영속화	Trust Calibration + 전문가 검토

다섯 사례의 공통점은 **자료가 누적되고 같은 질문이 반복되는 환경**이다. 자료가 정적이거나 일회성이라면 LLM Wiki의 가치는 제한적이다. 누적과 반복이 있어야 “정리해 두면 다음에 다시 정리하지 않아도 된다”는 핵심 메커니즘이 작동한다.

7. RAG와 LLM Wiki: 무엇이 무엇을 잘하는가

지금까지의 비교는 RAG의 가장 단순한 형태(Naive RAG)를 전제로 하였다. 그러나 RAG는 지난 2년간 빠르게 진화하였고, 현재는 “질문 시점에 조각을 검색해 결합하는” 단일 방식이 아니다. 정확한 비교를 위해서는 RAG 진영의 변형들을 먼저 정리할 필요가 있다.

7.1 RAG의 변형들 빠른 정리

- **Naive RAG** – chunk → embed → retrieve → generate. 가장 기본적인 형태이며, 앞서 비교한 방식이 이에 해당한다.
- **Advanced RAG** – 검색 전후를 최적화한다. 질의 재작성(query rewriting), 가설 답변 임베딩(HyDE), 재순위화(reranking) 등으로 검색 품질을 향상시킨다.
- **Modular RAG** – 검색, 라우팅, 메모리, 평가를 모듈로 분리해 조립한다. LlamaIndex나 LangGraph 같은 프레임워크가 이 패턴을 따르며, 도메인별로 다른 검색기를 사용하는 식의 유연성이 특징이다.

- **GraphRAG** – Microsoft가 공개한 그래프 기반 RAG. 텍스트에서 엔티티와 관계를 추출해 지식 그래프를 만들고, 커뮤니티 단위 요약까지 미리 생성한다. **여러 문서에 걸친 추론**과 **전체 코퍼스 요약**에 강점을 보인다.
- **Agentic RAG** – 에이전트가 검색, 도구 호출, 재추론을 능동적으로 조합한다. 답변이 부족하면 다시 검색하고, 다른 도구로 보강하는 반복 루프가 가능하다. 이 패턴은 다음 절에서 다루는 “Agentic AI”와 LLM Wiki의 통합과도 직접 연결된다.
- **Hybrid RAG** – BM25(키워드)와 벡터 검색을 함께 사용하고 reranker로 결합한다. 정확한 회수율과 의미 기반 검색을 동시에 확보하는 실용적 조합이다.

7.2 차원별로 보면 잘하는 영역이 다르다

다음 표에서 “차원”은 시스템을 평가하는 관점이며, “승자” 컬럼은 해당 차원에서 어느 쪽이 더 우위에 있는지를 명시한다. 차원의 정의가 모호하면 비교가 의미를 잃으므로, 각 행에 한 줄 정의를 부여하였다.

차원 (정의)	RAG의 동작	승자	LLM Wiki의 동작
신선도 새 자료 반영 속도	인덱스 갱신 즉시 반영 (분 단위)	RAG	위키 갱신과 검토에 시간 소요 (시간~일 단위)
대규모 정확 회수 수만 문서 정밀 검색	임베딩과 BM25로 정밀 검색	RAG	인덱스 파일 한계 (수천 페이지 이상에서 부족)
관계 추론 다단계 추론	GraphRAG가 엔티티 그래프로 강함	RAG (Graph)	사람이 만든 링크에 의존, 자동 추론 약함
누적 종합 결론을 한 페이지로 통합	매 질의마다 새로 종합 (휘발)	Wiki	종합 페이지가 영속화되어 다음 질의에 재사용
사람 가독성 검토 가능한 형태인가	답변이 채팅 기록에 묻힘	Wiki	마크다운 파일로 영구 보관, 검토 가능
긴 호흡 리서치 사고 흐름 추적	매 세션마다 처음부터 시작	Wiki	사고 흐름이 파일로 누적
콜드 스타트 즉시 사용 가능 여부	인덱싱 종료 후 즉시 사용 가능	RAG	위키가 정리될 때까지 가치 부족
운영 인프라 시작 복잡도	임베딩 DB, 벡터 검색 등 필요	Wiki	마크다운 파일과 Git만으로 시작
환각 위험 잘못된 답의 영속화	매번 검색하므로 누적되지 않음	RAG	잘못된 정리가 “공식 지식”으로 굳을 위험
출처 추적 인용 정밀도	chunk 단위 정확한 인용	RAG	페이지 단위, 원본 추적은 별도 관리
광범위 임의 질의 예측 못 한 질문 대응	인덱스만 있으면 즉답	RAG	정리되지 않은 영역은 답이 부실

승자 통계의 해석. RAG가 7개 차원, Wiki가 4개 차원에서 우세하다. 그러나 “승자 수”로 결론을 내리는 것은 의미가 없다. 중요한 것은 **조직이 어떤 차원에 가중치를 부여하는가**이다. 예컨대 “누적 종합”과 “사람 가독성”에 가중치를 두는 리서치 팀에서는 Wiki가 우위를 보이며, “대규모 정확 회수”에 가중치를 두는 사내 검색 팀에서는 RAG가 우위를 보인다.

표가 보여주는 또 다른 사실은 두 접근이 **서로 다른 시간 축에서 작동**한다는 점이다. RAG는 질문 시점의 동기 작업이고, LLM Wiki는 질문 사이의 비동기 작업이다. 동일 자원을 두고 경쟁하지 않으므로 **이론적으로는 보완 관계**가 성립한다.

7.3 보완 관계가 정말 성립하는가: 정직한 검토

“보완 관계”라는 표현은 너무 자주 사용되는 만큼 한 번 더 검증할 필요가 있다.

보완이 성립하는 근거

- **실패 모드가 서로를 보정한다.** RAG는 다중 문서 종합이 약하나 Wiki가 그 종합을 미리 만들어 둔다. 반대로 Wiki가 정리하지 못한 새로운 영역의 질문은 RAG가 원자료에서 직접 찾아낸다.
- **서로 다른 정보 밀도를 저장한다.** Raw는 고충실도 저구조, Wiki는 중충실도 고구조다. 손실 압축의 다른 단계이므로 한쪽이 다른 쪽을 대체할 수 없다.
- **Wiki의 stale 위험을 RAG가 보정한다.** Wiki가 옛 정보를 “공식 지식”으로 굳히려 할 때, 답변 생성 단계에서 raw 검색 결과와 비교함으로써 모순을 식별할 수 있다.
- **이미 업계 표준에 가깝다.** Notion AI, Glean, Confluence AI, Microsoft 365 Copilot은 모두 “원본 인덱스 + LLM이 만든 합성/요약 캐시”라는 동일한 패턴을 채택하고 있다.

무조건 보완은 아니라는 단서

- **소규모에선 둘 다 필요 없다.** 카파시 본인의 100건 규모 위키는 인덱스 파일과 텍스트 검색만으로 충분했다. 하이브리드는 두 파이프라인의 운영 비용을 정당화할 만큼의 규모와 질의 다양성이 있어야 가치가 난다.
- **Source of truth 충돌이 생긴다.** Wiki와 Raw가 다른 결론을 낼 때 어느 쪽을 신뢰할지에 대한 결정 규칙이 필요하다. 이 규칙 자체가 운영 부담이 된다.
- **Modular RAG가 성숙하면 일부 흡수된다.** 메모리 모듈, reranker, 캐시된 종합 답변 같은 기능이 결합되면 “별도 Wiki 계층”의 일부 역할을 RAG 안으로 끌어올 수 있다.

결론

보완 관계는 맞지만 **모든 규모에서 그런 건 아닙니다.** 진짜 가치가 나는 영역은 다음 세 조건이 겹칠 때입니다.

1. 자료가 수만 건 이상으로 커진다.
2. 질의가 “빠른 사실 회수”와 “깊은 종합”이 섞여 있다.
3. 지식이 장기간 누적되는 환경(대부분의 기업이 여기에 해당).

그러니까 “일단 둘 다 쓰자”가 아니라, 위 조건이 충족되는 시점에 하이브리드로 발전시키는 것이 합리적입니다.

7.4 하이브리드는 어떻게 흐르는가

조건이 맞을 때 두 파이프라인은 다음과 같이 맞물립니다.

1. **원본 자료 → RAG 인덱스** – 정확한 사실 회수와 chunk 단위 출처 인용을 위해.
2. **원본 자료 → LLM Wiki** – 자료를 통합하면서 위키 페이지를 자동 갱신. 이때 RAG 인덱스를 도구로 호출해 기존 위키와의 모순을 검출.

3. **LLM Wiki 페이지** → **RAG 인덱스** – 위키가 만든 종합 결과물을 다시 인덱스에 넣어, 검색 시 원자료뿐 아니라 정리본도 함께 참조.
4. **질의 라우팅** – 사실 회수성 질문은 RAG로, 종합/추론성 질문은 Wiki로, 모호하면 양쪽 결과를 합쳐서 답변.

핵심: LLM Wiki는 RAG의 대체가 아니라 **지식 유지보수 계층**입니다. RAG가 “질문 시점의 검색”을 담당한다면, LLM Wiki는 “질문 사이의 정리”를 담당합니다. 두 파이프라인이 서로의 출력을 입력으로 받아 순환하는 구조가 핵심이에요.

7.5 Agentic AI와 LLM Wiki: 다른 계층의 도구

지금까지 RAG 변형들과 비교했지만, 더 큰 비교 대상이 빠져 있습니다. 바로 **Agentic AI**예요. 최근 1~2년 사이 “에이전트”라는 단어가 업계 중심에 들어왔으니, LLM Wiki가 이 흐름과 어떤 관계인지 분명히 짚어야 합니다.

핵심 정답: Agentic AI와 LLM Wiki는 **경쟁 관계가 아니라 다른 계층의 도구**입니다. 더 정확히 말하면, Agentic AI 시스템에서 LLM Wiki를 한 도구로 호출할 수 있습니다.

개념 비교

차원	Agentic AI	LLM Wiki
주된 역할	작업 실행, 도구 호출, 다단계 추론	자료 정리, 색인, 종합
시간 축	순간 (작업 단위, 분~시간)	누적 (며칠~년)
출력	행동, API 호출, 코드 변경, 답변	마크다운 파일
메모리	작업 컨텍스트 (휘발성)	영속 외부 저장
관심사	“어떻게 작업을 완수할 것인가”	“무엇을 알고 있는가”
주요 위험	잘못된 행동 (API 호출 실패, 비용 폭발)	잘못된 정리 (hallucination 누적)

Agentic AI에서 LLM Wiki를 어떻게 사용하는가

에이전트는 도구를 호출하면서 작업을 수행합니다. 그중 “기억”에 해당하는 도구가 LLM Wiki입니다.

- **읽기(Retrieval)** – 에이전트가 작업을 시작할 때 위키에서 “이전에 비슷한 일을 한 적이 있나”를 조회. 컨텍스트로 활용.
- **쓰기(Persistence)** – 작업이 끝나면 결과, 발견한 패턴, 시행착오를 위키에 새 페이지로 저장. 다음 에이전트 세션이 그 페이지를 읽을 수 있게 함.

- **협업(Coordination)** – 여러 에이전트가 같은 위키를 공유하면, 한 에이전트가 만든 결과를 다른 에이전트가 이어받아 작업 가능.

실제 사례: Karpathy의 워크플로(Claude Code + Obsidian) 자체가 이미 “Agentic AI + LLM Wiki”의 초기 형태입니다. Claude Code가 에이전트 역할(작업 실행, 도구 호출), Obsidian이 LLM Wiki 역할(영속 저장, 사람 가독)을 합니다.

AWS Bedrock Agents에서의 통합 패턴

Bedrock Agents는 도구(Tool)를 등록해 에이전트가 호출할 수 있게 합니다. LLM Wiki를 도구로 등록하면 다음 흐름이 됩니다.

에이전트 작업 흐름 예시 (개발 작업 자동화)

1. 사용자: “payments 서비스의 재시도 로직을 개선해 줘”
2. 에이전트가 Wiki 도구 호출: `wiki.search("payments 재시도 로직")`
3. Wiki가 `decisions/payments-retry-policy.md` 페이지 반환 (과거 결정 근거)
4. 에이전트가 코드 도구로 변경 작성, 테스트 실행
5. 작업 완료 후 Wiki 도구 호출: `wiki.append("payments-retry-policy.md", "v2: 변경 내역과 근거")`
6. 변경 사항이 위키 PR로 자동 생성, 사람 검토 게이트로 들어감

통합 시 주의점

에이전트가 위키에 직접 쓰는 패턴은 강력하지만 위험도 따라옵니다.

- **Hallucination이 즉시 영속화된다** – 에이전트가 잘못된 결론을 위키에 쓰면, 그 다음 작업이 잘못된 위키를 신뢰 입력으로 받음. 5절의 누적 위험이 더 빠르게 진행됨.
- **쓰기 권한을 분리하라** – 에이전트는 “draft 페이지에 쓰기”만 허용하고, “운영 페이지로 머지”는 사람 또는 별도 검증 에이전트가 결정.
- **비용 폭발 위험** – 에이전트는 “읽기 → 추론 → 쓰기 → 다시 읽기”를 반복할 수 있어 토큰 사용량이 빠르게 늘어남. CloudWatch 비용 알림 필수.
- **여러 에이전트의 동시 쓰기 충돌** – 같은 페이지를 두 에이전트가 동시에 수정하면 PR 충돌. Git의 머지 전략이나 CodeCommit의 lock 메커니즘으로 해결.

핵심 메시지: Agentic AI를 도입하는 조직에 LLM Wiki는 “경쟁 패턴”이 아니라 **필수 보조 도구**입니다. 에이전트는 일을 하고, 위키는 그 일의 맥락과 결과를 보존합니다. 둘이 결합되면 “배우는 에이전트 시스템”에 한 발짝 가까워지지만, hallucination 영속화 위험도 같이 커지므로 권한 분리와 검토 게이트가 더욱 중요해집니다.

7.6 언제 무엇을 선택하나

- **LLM Wiki 단독으로 충분한 경우** – 자료 수백 건 이내, 같은 주제를 깊게 다루는 개인 리서치, 책 한 권을 읽으며 만드는 컴퍼니언 위키.
- **Modular/Hybrid RAG가 더 나은 경우** – 수만 건 이상의 문서, 빠르게 갱신되는 데이터(뉴스, 가격, 재고), 광범위한 임의 질의를 받는 사내 검색.

- **GraphRAG가 빛나는 경우** – 복잡한 관계 추론(“A 결정이 B 부서에 어떻게 영향을 줬나”)이나 전체 코퍼스에 대한 글로벌 요약이 필요할 때.
- **하이브리드가 최선인 경우** – 위 조건들이 섞인 대부분의 실제 운영 환경. 특히 기업 환경.

8. AWS로 LLM Wiki를 어떻게 만들 것인가

여기까지가 개념입니다. 그런데 개인 노트북의 마크다운 폴더를 넘어, **팀 단위** 또는 **전사 규모**로 LLM Wiki를 운영하려면 이야기가 달라집니다. 누가 자료를 넣어도 안전해야 하고, 권한을 분리해야 하고, 위키가 손상되면 복구할 수 있어야 하고, 비용은 통제 가능해야 하니까요.

AWS는 이 흐름의 각 단계에 잘 맞는 서비스를 갖추고 있습니다. 카파시의 원래 패턴을 그대로 유지하면서 **확장성, 보안, 거버넌스**를 더하는 방식입니다.

8.1 전체 아키텍처 한눈에 보기

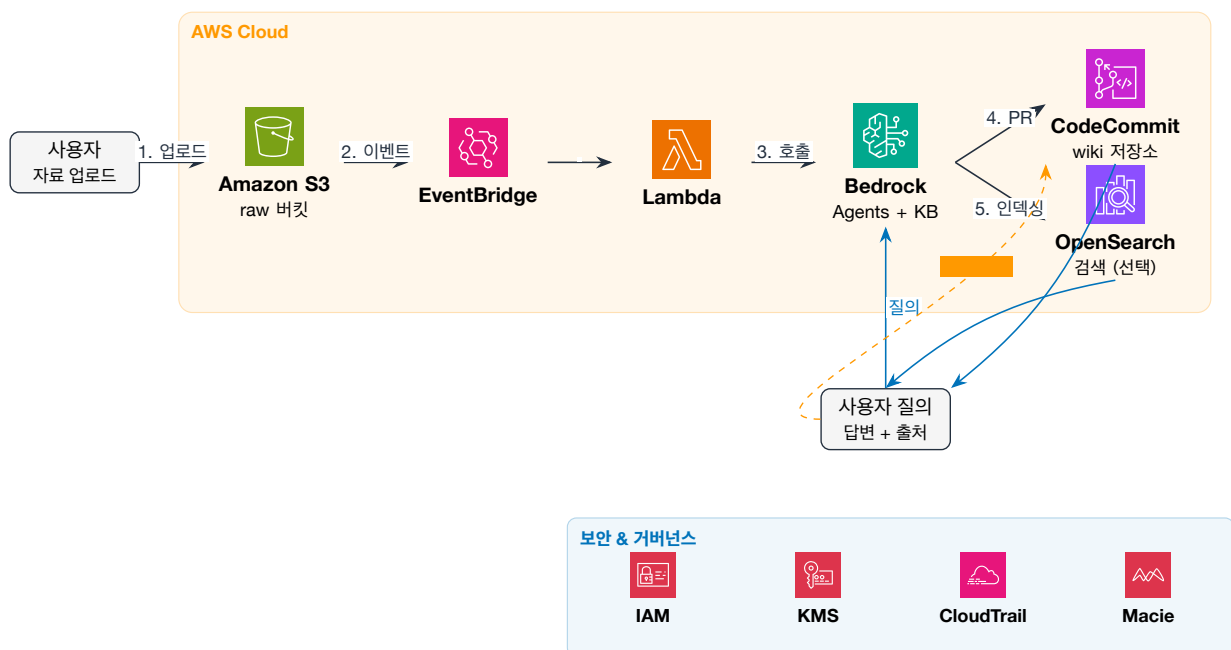


그림 1. Phase 1 — 기본 LLM Wiki 아키텍처. 자료가 들어오면 위키가 갱신되고, 질문은 위키 기반으로 답변됩니다. 검정 화살표는 Ingest, 파란 화살표는 Query, 주황 점선은 답변이 위키로 환원되는 피드백 루프. 자료 수백 건 규모, 단순한 질의로 출발할 때 적합합니다.

8.2 레이어별 매핑

원본 자료 저장: Amazon S3

역할: 불변(immutable)으로 유지해야 하는 원본 PDF, 회의록, 이미지, 코드 스냅샷의 영구 저장소.

핵심 기능:

- **S3 Object Lock** – 원본을 읽기 전용으로 잠가 LLM 에이전트가 실수로 덮어쓸 수 없게 합니다. “raw는 절대 수정하지 않는다”는 LLM Wiki의 원칙을 인프라 레벨에서 강제합니다.
- **S3 Versioning** – 자료가 갱신되더라도 모든 이전 버전이 남아 감사 추적이 가능합니다.
- **S3 Intelligent-Tiering** – 잘 안 읽히는 오래된 자료는 자동으로 저비용 계층으로 이동합니다.

위키 본체: AWS CodeCommit 또는 Amazon EFS

역할: LLM이 직접 쓰고 갱신하는 마크다운 파일 디렉토리.

선택지:

- **AWS CodeCommit + Git 워크플로** – 위키 자체를 Git 저장소로 두면 버전 관리, 브랜칭, PR 리뷰가 공짜로 따라옵니다. LLM이 만든 변경사항을 사람이 PR로 검토하고 머지하는 게이트를 자연스럽게 만들 수 있습니다.
- **Amazon EFS** – 여러 컨테이너나 Lambda가 동시에 읽고 쓰는 시나리오에 적합한 공유 파일 시스템.

추론 엔진: Amazon Bedrock

역할: Ingest, Query, Lint를 수행하는 LLM 자체.

핵심 기능:

- **모델 선택권** – Claude(Anthropic), Llama(Meta), Mistral, Amazon Nova 등을 단일 API로 전환 가능. 모델 락인 없이 비용과 품질 트레이드오프를 조정할 수 있습니다.
- **Bedrock Agents** – LLM이 S3 읽기, 위키 갱신, 외부 검색 같은 도구를 호출하는 에이전트 워크플로를 그대로 정의할 수 있습니다.
- **Knowledge Bases for Bedrock** – 위키 규모가 커지면 임베딩 기반 검색을 추가 인프라 없이 붙일 수 있습니다.
- **Guardrails** – 민감 정보 마스킹과 유해 콘텐츠 차단을 정책으로 강제합니다.

자료 유입 자동화: AWS Lambda + EventBridge

역할: 새 자료가 들어오면 자동으로 Ingest 워크플로를 작동시키는 트리거.

시나리오: 사용자가 S3 raw/ 버킷에 PDF를 업로드하면, S3 이벤트가 Lambda를 호출하고, Lambda가 Bedrock 에이전트를 깨워 위키 갱신 PR을 생성합니다. 사람은 PR만 검토하면 됩니다.

검색 계층: Amazon OpenSearch / Kendra

역할: 위에서 본 RAG와 LLM Wiki 하이브리드 패턴의 핵심 구성요소. 위키만 운영해도 수천 페이지를 넘으면 인덱스 파일만으로는 한계가 옵니다.

- **Amazon OpenSearch Service** – BM25와 벡터 하이브리드 검색. Hybrid RAG 패턴을 그대로 클라우드 규모로 운영합니다. 카파시가 개인용으로 쓴 qmd의 엔터프라이즈 버전이라고 보면 됩니다.
- **Amazon Kendra** – 자연어 질의에 강한 엔터프라이즈 검색. 사내 권한과 결합된 결과를 제공합니다.
- **Knowledge Bases for Bedrock** – Modular RAG에 가까운 구성으로, 임베딩 검색을 Bedrock과 자연스럽게 묶어 줍니다.

보안과 거버넌스: IAM, KMS, CloudTrail, Macie

역할: 팀과 기업 환경에서 가장 중요한 부분.

- **IAM** – LLM 에이전트의 읽기와 쓰기 권한을 분리. 원본은 read-only, 위키는 PR 제안만, 운영 페이지는 사람 승인 후 머지.
- **AWS KMS** – 모든 자료를 자체 키로 암호화. 회사 데이터가 어떤 모델에도 학습되지 않게 보장함

니다.

- **AWS CloudTrail** – LLM이 어떤 파일을 언제 읽고 어떻게 바꿨는지 전체 감사 로그.
- **Amazon Macie** – 원본에 PII나 기밀 정보가 섞여 있으면 자동 탐지.

8.3 전체 흐름

Ingest 흐름 예시

1. 사용자가 새 회의록 PDF를 **S3 raw 버킷**에 업로드
2. **S3 이벤트**가 **Lambda**를 트리거
3. Lambda가 **Bedrock Agent**에게 “이 자료를 위키에 통합하라” 지시
4. 에이전트가 PDF를 읽고 **Knowledge Base**로 관련 위키 페이지 검색
5. 갱신할 페이지들의 변경 사항을 **CodeCommit**에 PR로 생성
6. Slack이나 이메일로 사람에게 검토 알림 (**SNS**)
7. 사람이 PR 머지, 위키 최신화 완료
8. 모든 단계가 **CloudTrail**에 기록

8.4 Phase 2 — RAG와 LLM Wiki를 결합한 하이브리드 아키텍처

자료가 수만 건을 넘어가고 질의의 종류가 다양해지면 Phase 1만으로는 부족합니다. 앞서 본 “보완 관계” 분석을 실제 AWS 서비스 위에 펼쳐 놓으면 다음 그림처럼 됩니다.

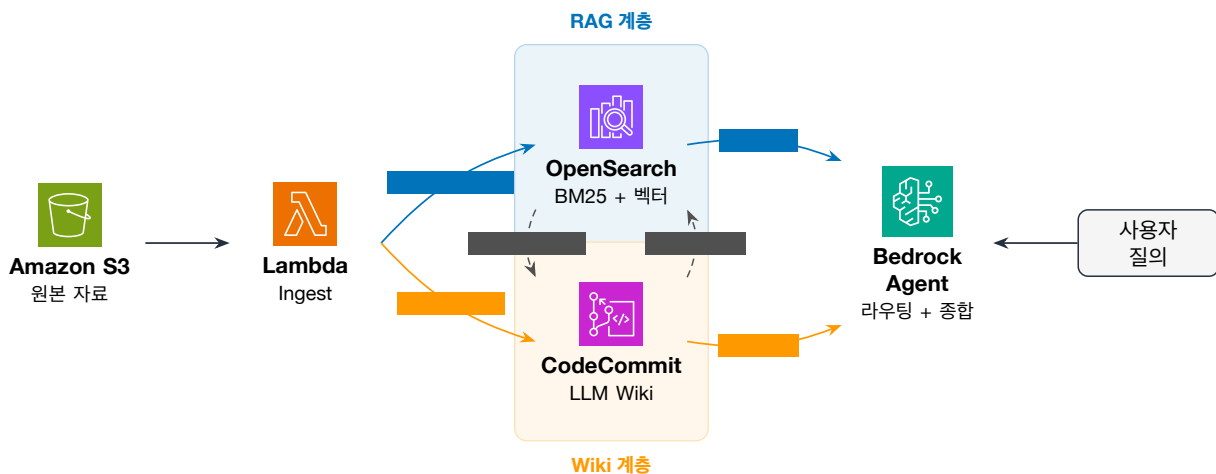


그림 2. Phase 2 — RAG와 LLM Wiki 하이브리드 아키텍처. 같은 원본 자료가 두 파이프라인(RAG 인덱스, LLM Wiki)으로 동시에 흘러 들어가고, 두 계층이 서로의 출력을 참조하며 순환합니다. 질의가 들어오면 Bedrock Agent가 라우팅합니다. 사실 회수성 질문은 RAG로, 종합과 추론이 필요한 질문은 Wiki로 보냅니다. 둘 다 필요한 경우에는 양쪽 결과를 합쳐 답변합니다. 점선 화살표(③, ④)는 두 계층이 서로를 검증하고 강화하는 cross-pipeline 흐름입니다.

핵심 흐름 요약

- ① **RAG 인덱싱** – Lambda가 원본을 chunk로 쪼개 OpenSearch에 임베딩과 함께 저장. 출처 단위로 정확히 추적.
- ② **Wiki 통합** – 같은 Lambda가 Bedrock Agent를 호출해 위키 페이지를 갱신. 새 자료가 어떤 페이지를 건드려야 하는지 에이전트가 결정.

- ③ **정리본을 인덱스에 환원** – 위키가 만든 종합 페이지(비교, 분석, 결론)도 OpenSearch에 다시 색인. 검색 결과에 “원자료”와 “정리본”이 함께 노출됨.
- ④ **모순 검출** – 위키 갱신 시 에이전트가 OpenSearch를 도구로 호출해 “기존 위키와 새 자료가 충돌하는지” 점검. 충돌이 발견되면 PR에 명시.
- **질의 라우팅** – Bedrock Agent가 질문 유형을 판단. “2024년 3월 매출은?” 같은 사실 회수성 질문은 RAG, “우리가 그 결정을 왜 했지?”는 Wiki, 둘 다 필요하면 두 결과를 함께 종합.

Phase 1과 무엇이 달라지나

- OpenSearch가 **선택 사항이 아닌 필수**가 됩니다. RAG 계층의 본체가 여기에 있어요.
- Bedrock Agent의 역할이 **단순 종합에서 라우팅 + 종합**으로 확대됩니다. 어떤 도구를 어떤 순서로 호출할지 결정하는 책임이 추가됩니다.
- 두 계층이 **서로의 도구**가 됩니다. Wiki는 RAG로 모순을 검사하고, RAG는 Wiki 페이지를 검색 결과에 포함합니다.
- 거버넌스 부담이 **두 배**로 늘어납니다. 인덱스 갱신 정책, 위키 PR 정책, 두 결과의 우선순위 등 운영 규칙이 필요해집니다.

언제 Phase 2로 가야 하나

다음 신호가 둘 이상 보이면 Phase 2 전환을 검토할 시점입니다.

- Wiki만으로는 “모르는 영역”에서 답이 부실하다는 피드백이 늘어남.
- 자료가 수천 건을 넘어가면서 Bedrock에 위키 전체를 넣을 수 없게 됨.
- “출처를 chunk 단위로 보여달라”는 컴플라이언스 요구가 생김.
- 빠르게 갱신되는 데이터(가격, 재고, 인사 정보)가 위키에서 자주 stale 상태가 됨.

8.5 운영 모니터링: CloudWatch 대시보드 가이드

LLM Wiki를 “실험”에서 “인프라”로 격상시키려면 운영 데이터를 봐야 합니다. CloudWatch 대시보드에 다음 지표를 시각화하세요.

지표 그룹	메트릭 이름 (예시)	무엇을 보는가
사용량	Bedrock Invoke-Model count, S3 PutObject count	도입 활성화도, 위키 성장 속도
비용	Bedrock 토큰 사용량, OpenSearch 노드 비용	예산 대비 추세, 모델별 비용 분포
지연	Lambda Duration p95, Bedrock Latency	사용자 경험, SLA 위반 가능성
오류	Lambda Errors, Bedrock ThrottlingException	시스템 안정성
품질 (커스텀)	Faithfulness 평균, Drift Rate, PR rejection 비율	LLM 작업 품질 추세
거버넌스	CloudTrail 이벤트 수, IAM 권한 변경 수	보안 이상 징후

권장 알람 설정 (CloudWatch Alarms):

- 비용 알람 – 일별 Bedrock 비용이 예산의 130%를 초과하면 알람.
- 품질 알람 – Faithfulness가 임계값(85%) 미만으로 3일 연속 떨어지면 알람.
- Drift 알람 – Wiki 페이지 중 “stale 표시”(검토 주기 만료) 비율이 10%를 넘으면 알람.
- 보안 알람 – IAM 정책이 변경되거나 raw 버킷에 쓰기 시도가 발생하면 즉시 알람.
- 사용 패턴 알람 – 주간 질의 수가 전주 대비 50% 이상 급증/급감하면 알람 (사용 패턴 변화 감지).

이 대시보드를 매주 5분만 봐도 “이 시스템이 살아있는지”를 즉시 알 수 있습니다. 운영 단계로 격상되는 시스템에는 반드시 이 정도의 모니터링이 따라와야 합니다.

9. 어떤 상황에서 무엇을 쓸 것인가

여기까지 RAG 변형, LLM Wiki, 하이브리드 아키텍처를 모두 봤습니다. 그런데 “우리 조직은 어디서 시작해야 하나?”라는 실제 의사결정은 또 다른 문제예요. 이 섹션은 그 결정을 돕기 위한 두 가지 도구를 제공합니다.

9.1 의사결정 트리: 5분 안에 출발점 찾기

복잡한 비교 표를 다 읽지 않아도 다음 흐름을 따라가면 시작점이 나옵니다.



그림 3. 5분 의사결정 트리. 두 개의 질문 — “자료 규모”와 “질의 특성” — 으로 출발점을 좁힙니다. 어느 분기에서 시작하든 12~18개월 후에는 결국 Phase 2 하이브리드로 수렴하는 경우가 많습니다.

보충 가이드

- 위 트리는 출발점일 뿐, 어느 분기에서든 12~18개월 후에는 결국 **Phase 2 하이브리드**로 수렴하는 경우가 많습니다. 처음부터 모든 걸 만들지 말고, 시작 지점을 정한 뒤 신호를 보고 확장하는 것이 정석입니다.
- “복잡한 관계 추론”의 기준이 모호하면 다음 질문으로 판단하세요. “A 결정이 B 부서, C 프로젝트, D 고객에 어떻게 연쇄적으로 영향을 줬는지”와 같은 질문이 자주 나오는가? 그렇다면 GraphRAG가 빛납니다.

9.2 시나리오 매트릭스: 비슷한 사례 찾기

자기 상황과 가장 가까운 시나리오를 찾아 추천 조합과 시작점을 참고하세요.

시나리오	자료 규모	핵심 질의 패턴	추천 조합	시작점
개인 리서치 / 박사 과정	수백 건	깊은 종합, 긴 호흡	LLM Wiki 단독	Obsidian + Bedrock, Phase 1
스타트업 사내 위키	수백~수천 건	온보딩, 의사결정 추적	LLM Wiki + 가벼운 RAG	Phase 1로 시작, 6개월 후 Phase 2
대기업 컴플라이언스 문서	수만 건 이상	chunk 단위 출처 인용	Modular RAG 우선	OpenSearch + Kendra, Wiki는 후순위
컨설팅 펌 / 로펌	중~대 규모	다중 클라이언트 격리 + 종합	Hybrid (Phase 2)	클라이언트별 권한 분리 우선
연구 그룹 / 인텔리전스	중규모	엔티티 관계 추론	GraphRAG + Wiki	GraphRAG로 시작, Wiki는 보조
미디어 / 뉴스룸	수만 건 / 일일 갱신	신선도 절대	RAG 우선	OpenSearch만으로 운영
의료 / 임상 자료	중~대 규모	정확성 + 감사 추적	Hybrid + 강한 거버넌스	Phase 2 + Macie + 고도화된 IAM
독서 / 학습 노트	수십~수백 건	책 한 권 깊이 이해	LLM Wiki 단독	Obsidian만으로도 충분

9.3 어떤 직군이 어떻게 쓰는가

조직 안에서 누가 주로 사용하는지에 따라 인터페이스 선택도 달라집니다.

- **개발자/엔지니어** – 터미널에서 claude나 Codex 직접 호출. CodeCommit Git 워크플로 그대로 사용. 마크다운을 IDE에서 봄.
- **기획자/PM** – Bedrock Agent를 감싼 Slack 봇이나 웹 UI. 직접 마크다운을 만지지 않고 “질문하고 답 받는다” 패턴.
- **경영진/임원** – 주간 자동 요약 리포트(Wiki의 “Lint” 결과 요약). SES로 메일 발송하거나 QuickSight 대시보드.
- **고객 지원/CS** – Kendra 기반 챗봇. 사내 위키와 RAG 인덱스를 함께 검색. 답변에 출처 명시 필수.
- **컴플라이언스/감사** – CloudTrail 로그와 위키 PR 히스토리에 직접 접근. “누가 언제 무엇을 바꿨는가”에 답할 수 있어야 함.

같은 위키라도 직군별로 **다른 인터페이스**를 붙여 주는 게 채택률에 큰 차이를 만듭니다. 마크다운 파일 자체에 익숙하지 않은 직군에게는 “파일”을 보여주지 마세요.

9.4 데이터 특성으로 본 선택 가이드

데이터 특성	권장 접근	이유
구조화 (DB, 테이블)	RAG 보다 Text-to-SQL 또는 직접 SQL	정확도가 검색보다 훨씬 높음
반구조화 (JSON, 로그)	RAG (스키마 인식 chunking)	필드 단위 검색이 가능
비정형 텍스트	RAG + LLM Wiki 둘 다	가장 일반적 케이스
이미지/도표 포함	멀티모달 RAG + Wiki에 캡션 페이지	Wiki에는 “이 도표는 X를 보여준다”식 메타정보
빠른 갱신 (분/시간)	RAG 우선	Wiki 갱신 비용을 정당화 못함
느린 갱신 (주/월)	LLM Wiki 우선	종합의 가치가 비용보다 큼
민감/규제 데이터	Hybrid + 권한 분리	권한별 인덱스 분리, 위키는 사람 검토 게이트

9.5 혼한 안티패턴

선택 가이드만큼 중요한 게 “하지 말아야 할 패턴”입니다.

- **모든 자료를 하나의 위키에 욱여넣기.** 도메인이 너무 다른 자료(개인 메모 + 회사 자료 + 클라이언트 자료)를 한 위키에 모으면 권한과 종합 모두 망가집니다. 도메인별로 분리하세요.
- **Phase 2부터 시작하기.** 검증되지 않은 가치에 두 파이프라인의 운영 부담을 떠안으면 도입이 멈춥니다. Phase 1으로 시작해 신호를 본 뒤 확장하세요.
- **Wiki 결과를 출처 검증 없이 외부에 인용.** LLM Wiki는 “2차 산물”입니다. 외부 보고서나 의사결정 근거로 쓸 때는 반드시 원본을 확인하세요.
- **Lint 없이 운영.** 위키가 시간이 지나면 stale 페이지가 쌓입니다. 주 1회는 Bedrock Agent로 모순/고아 페이지 점검을 자동화해야 합니다.
- **모델을 미리 고정하기.** “Claude로 가자/GPT로 가자”를 6개월간 못 바꾸도록 잠가두면 비용 최적화 기회를 잃습니다. Bedrock의 모델 추상화를 이용해 에이전트 단위로 모델을 갈아끼울 수 있게 설계하세요.

10. 비판적 검토와 한계: 무엇을 조심해야 하는가

지금까지의 글은 LLM Wiki의 가능성에 무게를 실었습니다. 그러나 이 패턴을 도입하기 전에 **반드시 인지해야 할 한계**가 있습니다. 이 섹션은 그 한계를 정직하게 정리합니다. 옹호 일변도의 자료는 의사결정에 도움이 되지 않으니까요.

10.1 LLM은 “지식”을 만들지 않는다

본문에서 “LLM이 지식을 정리한다”, “기억할 자리를 만든다” 같은 표현을 썼지만, 엄밀히 말하면 부정확합니다. **LLM은 텍스트를 압축 변환할 뿐, 의미적 이해나 추론을 수행하는 게 아닙니다.**

- **“기억”이 아니라 “기록”** – 마크다운 파일은 외부 기록 매체이지, 모델의 기억이 아닙니다. 사람의 기억은 분산 표현으로 형성되지만 위키는 손실 압축된 텍스트일 뿐입니다.
- **“사서”라는 비유의 한계** – 사서는 자기가 정리한 책의 내용에 책임을 집니다. LLM은 책임지지 않습니다. 위키 페이지가 틀렸을 때 “어디서 잘못됐는지”를 LLM 자신은 알지 못합니다.
- **모순 검출의 한계** – LLM은 명제 논리를 수행하지 않습니다. 표면적 텍스트 유사성에 의존할 뿐이고, 실제 의미적 모순(예: “X는 A다”와 “X는 A가 아닐 수 있다”)을 잡아내는 능력은 제한적입니다.

실무적 함의: LLM Wiki는 “지식 시스템”이 아니라 **사람이 읽을 수 있는 캐시된 텍스트 요약**으로 정의하는 것이 정직합니다. 이 정의를 받아들이면 도구 사용법도 명확해집니다. 위키는 **탐색 보조**이지, 의사결정의 1차 근거가 아닙니다.

World model이 필요한 영역에서의 한계

- LLM Wiki는 본질적으로 **텍스트를 텍스트로 변환하는 시스템**이지, 물리적·시간적·인과적 reasoning을 수행하지 않습니다.
- 다음 영역에서는 LLM Wiki만으로 부족합니다:
 - 로봇틱스 운영 매뉴얼 – 물리적 결과 예측 필요
 - 인과 분석 – “이 결정이 그 결과의 원인인가?”에 대한 판단
 - 시뮬레이션 결과 종합 – 시간 진화에 대한 추론
 - 공정 안전 분석 – 반사실(counterfactual) 추론
- 이런 영역에서는 LLM Wiki를 “기록 도구”로만 쓰고, 판단은 도메인 전문가가 직접 수행해야 합니다.

10.2 Hallucination 누적의 위험

가장 위험한 실패 모드는 **잘못된 정리가 시간이 지나면서 “공식 사실”로 굳는 현상**입니다.

발생 메커니즘:

1. Wiki v1에서 LLM이 미묘한 사실 오류를 만든다 (예: 날짜를 한 자리 잘못 적음).
2. Wiki v2에서 새 자료를 통합할 때, LLM은 기존 위키를 신뢰할 만한 입력으로 간주한다.
3. v2가 v1의 오류를 강화하거나, 그 오류 위에 새 내용을 쌓는다.
4. Wiki v3쯤 되면 그 오류는 “출처”까지 그럴듯하게 붙은 “공식 사실”로 굳어진다.
5. 사람은 그것을 의심하지 않고 의사결정에 사용한다.

이 문제가 RAG보다 LLM Wiki에서 더 심각한 이유: RAG는 매번 원본을 다시 검색하기 때문에 누적되지 않습니다. LLM Wiki는 정의상 누적 구조이므로, 누적되는 것이 사실인지 오류인지에 운명이 달려 있습니다.

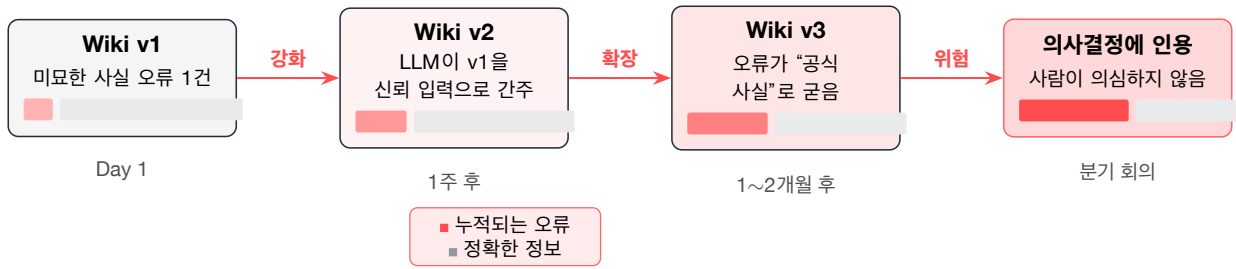


그림 4. Hallucination 누적의 4단계 시각화. 시간이 지나면서 빨간 영역(오류)이 확장되고, 동시에 “출처”가 그럴듯해지면서 사람의 의심 임계값을 통과합니다. 위키가 누적 구조라는 강점이 곧 이 위험의 원인입니다.

대응:

- **Trust level 명시** – 모든 위키 페이지에 “원본 직접 인용” / “LLM 종합” / “LLM 추론” 세 단계로 신뢰도를 표시.
- **원본 fingerprint 보존** – 위키 페이지의 모든 주장에 출처 자료의 정확한 인용(quote)을 함께 저장. 정기적으로 인용이 원본과 일치하는지 자동 검증.
- **인간 검토 게이트를 “권장”이 아니라 “필수”로** – 특히 의사결정에 인용될 페이지는 사람 승인 없이 운영 영역으로 머지되지 않도록 IAM과 PR 정책으로 강제.

Trust Calibration 워크플로

Trust level 표시만으로는 충분하지 않습니다. 어떤 페이지에 어떤 권한과 검토 빈도를 부여할지 명시적인 매트릭스가 필요합니다.

	기본 신뢰도	검토 주기	권한 정책
페이지 카테고리			
원본 직접 인용	높음	연 1회	LLM 자동 머지 가능
LLM 종합	중간	분기 1회	사람 PR 승인 필수
LLM 추론 (가설)	낮음	월 1회	“Draft” 상태로만 유지, 인용 금지
의사결정 근거	매우 높음	도입 전 + 분기	도메인 전문가 + 컴플라이언스 2단계 승인
컴플라이언스 답변	매우 높음	사용 직전 매번	사용 시점 자동 검증 + 사람 승인

운영 팁: 페이지 frontmatter에 `trust_level: <high|medium|low>`와 `review_due: <date>` 필드를 명시하면, Lambda + EventBridge로 자동 알림을 만들 수 있습니다. 검토 주기가 지나면 페이지가 자동으로 “stale” 표시되고 사용자에게 경고가 표시됩니다.

10.3 측정 가능성: “잘 작동한다”를 무엇으로 판단하는가

기존 90일 로드맵의 게이트는 정성적 질문 위주였습니다. 더 엄밀한 운영을 위해서는 **자동 평가 메트릭**이 필요합니다.

핵심 메트릭 4가지 (RAGAS와 유사 프레임워크 기준)

- **Faithfulness (충실도)** – 위키 페이지의 주장이 원본 자료에 실제로 근거하는가? LLM-as-a-Judge로 주 1회 샘플링해 측정.
- **Answer Relevance (답변 관련성)** – 사용자 질문에 대한 답변이 질문의 의도에 부합하는가?
- **Context Precision (컨텍스트 정밀도)** – RAG가 검색한 chunk 중 실제로 답에 사용된 비율.
- **Drift Rate (이탈률)** – 위키 페이지가 시간이 지나면서 원본과 얼마나 멀어졌는지. 정기 자동 검증.

운영 권장:

- Day 30 게이트부터 **Faithfulness 점수 90% 이상**을 통과 조건으로 추가.¹
- Day 60 이후 **Drift Rate 5% 미만**² 유지를 운영 SLA로 설정.
- LLM-as-a-Judge 평가는 다른 모델(예: 위키 운영은 Claude, 평가는 Nova) 조합으로 실행해 자기 검증 함정을 피합니다.

10.4 실패 비용 추정: 잘못된 정리가 의사결정에 들어갔을 때

LLM Wiki 도입 ROI를 계산할 때 **실패 시나리오의 비용**이 빠지면 의사결정이 한쪽으로 기울입니다.

실패 유형	발생 가능성	잠재 손실
경미한 사실 오류가 보고서에 인용	중~높음	평판/신뢰도 손상, 재작업 비용
중요한 결정의 근거가 stale 정보	중간	잘못된 투자, 기회비용
컴플라이언스 답변에 hallucination 포함	낮음~중간	규제 위반, 벌금, 소송
민감 정보가 위키에 부지불식간 노출	낮음	보안 사고, 데이터 유출
모순된 결론이 부서 간 갈등 유발	중간	협업 효율 저하

ROI 재계산 원칙: 기대 가치 = (성공 시 절감 효과 × 성공 확률) – (실패 시 손실 × 실패 확률). 이 식에서 “실패 확률”을 0으로 가정하지 마세요. 자동 평가 메트릭이 있어야 이 확률을 의미 있게 추정할 수 있습니다.

¹이 임계값은 RAGAS 프레임워크의 일반적 권장 출발점에 기반한 보수적 추정치입니다. 도메인에 따라 의료/법률은 95%, 일반 사내 위키는 85%가 적절할 수 있습니다. 자기 조직의 도메인 위험도에 맞춰 조정하세요.

²Drift Rate 5%는 “원본과의 의미적 거리가 측정 가능한 수준에서 안정적”을 의미하는 운영 SLA 출발점입니다. 학술 표준이 아닌 실무 권장값입니다.

10.5 LLM Wiki가 적합하지 않은 경우

선택 가이드에서 “적합한 경우”를 다뤘으니, 반대로 **도입을 권장하지 않는 경우도** 명시합니다.

- **고위험 의사결정 영역** – 의료 진단, 법적 판단, 재무 보고서 자동 생성 등. 인간 검토를 거치지 않은 LLM Wiki 출력은 1차 근거로 쓰면 안 됩니다.
- **진실 일관성이 절대적인 도메인** – 회계 원장, 인사 기록, 약품 처방 데이터 등. 이런 영역은 정형 DB가 source of truth여야 하고, LLM Wiki는 색인 보조에 그쳐야 합니다.
- **빠른 갱신 + 낮은 모순 허용** – 가격 정보, 재고, 실시간 운영 지표. 위키 갱신 지연 사이에 stale 정보가 의사결정에 쓰일 위험이 큼니다.
- **소규모(자료 50건 이하)** – LLM 호출 비용과 큐레이션 시간이 가치를 만회하지 못합니다. 텍스트 검색만으로 충분합니다.
- **권한 분리가 복잡한 다중 클라이언트 환경** – 컨설팅 펌처럼 클라이언트별 정보 격리가 엄격해야 하는 경우, 단일 위키 모델은 위험합니다. 클라이언트별 별도 위키와 강한 IAM 분리가 전제되어야 합니다.

10.6 기억과 가중치: Fine-tuning이라는 다음 질문

본문은 LLM Wiki를 “외부 텍스트 파일에 저장된 캐시”로 다뤘습니다. 그런데 위키 페이지가 충분히 쌓이면 자연스럽게 따라오는 질문이 있습니다. 이 코퍼스로 작은 모델을 *fine-tune*하면 지식이 가중치 안으로 들어가지 않을까?

Karpathy 자신도 후속 글에서 이 가능성을 언급했습니다. 합성 데이터 생성과 fine-tuning을 결합하면, “컨텍스트로 매번 읽는 방식”에서 “가중치 안에 반영되는 방식”으로 진화할 수 있다는 것입니다.

현 시점에서의 입장:

- Fine-tuning은 “다음 단계의 가능성”이지 **현재 핵심 권장 사항이 아닙니다**.
- 위키가 안정화되고 평가 메트릭이 일정 수준 이상에서 유지될 때까지는 외부 파일 + RAG + 위키 조합이 더 안전합니다.
- Fine-tuning은 모델 가중치 안의 hallucination을 **검증과 롤백이 더 어려운** 형태로 만든다는 점을 기억해야 합니다. 위키는 PR로 되돌릴 수 있지만 fine-tuned 모델은 그렇지 않습니다.
- AWS Bedrock의 Custom Model Import나 Bedrock Fine-tuning을 검토할 시점은, 위키 운영 1~2년 후, 도메인 데이터가 충분히 정제된 다음입니다.

핵심: LLM Wiki는 **기억층이 아니라 외부 캐시 색인**입니다. “기억”이라는 표현이 본문 앞부분에 등장한 건 비유의 편의를 위한 것이며, 시스템의 실제 동작은 “LLM이 마크다운 파일을 읽고 쓰는 것”에 그칩니다. 이 경계를 명확히 인식한 채로 도입하면, 도구의 가치를 누리면서 한계로 인한 사고도 피할 수 있습니다.

11. 비즈니스 가치 분석

기술적으로 가능하다는 것과 비즈니스적으로 의미가 있다는 것은 다릅니다. AWS 위에서 LLM Wiki를 운영했을 때 실제로 어떤 가치를 만들 수 있는지 짚어봅니다.

11.1 개인이나 소규모 팀 관점

- **초기 비용 거의 0** – S3와 Bedrock의 사용량 기반 과금. 자료 100개, 위키 수백 페이지 규모면 월 수만 원 수준입니다.

- **제로 인프라 운영** – 서버 관리, 백업, 패치를 직접 안 해도 됩니다. 마크다운 파일과 프롬프트에만 집중할 수 있습니다.
- **데이터 주권** – Bedrock에 들어간 데이터는 모델 학습에 쓰이지 않습니다. ChatGPT 무료 플랜에 자료를 올리는 것과 본질적으로 다릅니다.

11.2 기업 관점: 수치로 보는 효과

LLM Wiki가 풀어주는 가장 큰 비용은 **지식 유지보수 인건비**와 **지식 손실 비용**입니다. 아래 수치는 공식 검증된 벤치마크가 아니라 도입 초기 측정 가이드용 추정치입니다. 도입 후 자기 조직의 데이터로 재측정해야 합니다.

영역	기존 방식	LLM Wiki on AWS	근거
신규 입사자 온보딩	평균 2~4주, 시니어 시간 소모	정리된 위키로 1주 이내 단축	추정치
반복 질문 응대	시니어가 매번 직접 답변	위키 자동 답변, 시니어 개입 최소화	추정치
의사결정 추적	PR이나 회의록 수동 검색	출처 인용한 답변으로 즉시 회수	정성적
장애 회고 활용	작성 후 사실상 사장	새 장애 발생 시 자동으로 유사 사례 추천	정성적
문서 유지보수 부담	누구도 안 하고 싶어함	LLM이 기계 작업 대부분, 사람 검토	추정치

측정 권장: 도입 전 4주간 베이스라인을 기록(시니어가 반복 질문에 답하는 시간, 신규 입사자가 “기본 시스템 구조”를 이해하는 데 걸리는 시간)하고, Day 90 시점에 같은 지표를 재측정해 비교하세요. 이 차이가 ROI의 진짜 근거입니다.

11.3 AWS를 선택할 때의 고유 가치

오픈소스 마크다운과 로컬 LLM 조합으로도 LLM Wiki를 만들 수 있습니다. 그럼에도 AWS를 선택하는 합리적 이유는 다음과 같습니다.

- **모델 선택의 자유** – Bedrock은 단일 API로 Claude, Llama, Nova, Mistral 등을 전환합니다. 어느 모델이 가장 좋을지 미리 결정하지 않아도 됩니다.
- **기존 데이터와의 결합** – 회사 자료가 이미 S3, RDS, Redshift에 있다면, 별도 마이그레이션 없이 그 자리에서 위키 소스로 쓸 수 있습니다.
- **컴플라이언스 기본 탑재** – HIPAA, GDPR, ISO 27001, SOC 2 등을 충족하는 환경에서 시작합니다. 금융, 의료, 공공처럼 규제가 강한 산업에서 “일단 시작”이 가능합니다.
- **확장의 부드러움** – 개인 실험에서 팀 위키, 전사 지식 베이스로 가는 길에 아키텍처를 갈아엎을 필요가 없습니다. 같은 S3, 같은 Bedrock 위에서 권한과 도구만 늘리면 됩니다.

11.4 Bedrock 락인 우려와 Self-hosted 옵션

“Bedrock에서 시작하라”는 권고와 “모델 락인을 피하라”는 권고는 일견 충돌합니다. 정직하게 말하면 **Bedrock도 일종의 추상화 락인**입니다. AWS 외부에서는 같은 워크플로를 쉽게 옮길 수 없으니까요. 이 우려에 대한 균형 잡힌 답은 다음과 같습니다.

선택지	Bedrock 사용	Self-hosted (예: SageMaker JumpStart + Llama)
시작 비용	낮음 (사용량 기반)	중간~높음 (GPU 인스턴스 운영)
운영 부담	낮음 (관리형)	높음 (모델 배포, 스케일링 책임)
모델 선택 폭	7~10개 (지속 추가)	오픈 가중치 모든 모델
데이터 주권	강함 (학습 비사용)	매우 강함 (자체 인프라)
이식성	낮음 (Bedrock API 의존)	높음 (어떤 클라우드/온프레미스로도 이전 가능)
적합한 경우	빠른 시작, 운영팀 작음	장기 운영, 모델 fine-tuning, 극단적 데이터 통제

권장 경로: Bedrock으로 검증 단계를 빠르게 통과한 뒤, Phase 2 운영화 시점에서 self-hosted 옵션을 함께 검토합니다. 두 옵션은 LiteLLM 같은 모델 추상화 레이어를 두면 코드 변경 없이 전환할 수 있습니다.

11.5 90일 도입 로드맵: 검증 → 확장 → 운영화

거창한 시스템부터 만들 필요 없습니다. “작게 시작해 신호를 보고 확장한다”는 원칙을 90일 단위로 펼치면 다음과 같습니다.

목표: “우리 조직에서 LLM Wiki가 가치를 만드는가”를 증거로 확인.

- **1주차** – S3 버킷 두 개(raw/, wiki/)와 Bedrock 액세스 활성화. 한 명이 자기 자료 20~50건 업로드.
- **2주차** – Lambda + EventBridge로 자동 Ingest 흐름 구성. AGENTS.md 스키마 작성.
- **3주차** – CodeCommit Git화. PR 검토 게이트 도입. 매일 Wiki를 한 번씩 읽으며 품질 평가.
- **4주차** – 첫 번째 측정 지점. 다음 4가지 지표 기록.

30일 차 게이트 점검:

1. Wiki 페이지에서 “이 정보를 다시 찾기 위해 원본을 안 봐도 됐다”는 경험이 5번 이상 있는가?
2. LLM이 만든 페이지 중 “명백히 틀린” 것이 10% 이하인가?
3. 자동 평가 메트릭상 **Faithfulness 점수가 90% 이상**인가? (LLM-as-a-Judge로 샘플 20건 측정)
4. 자료 추가 → Wiki 갱신 자동화가 안정적으로 작동하는가?
5. 매주 위키 읽는 데 30분 미만이 드는가?

3개 이상 “예”면 Day 31~60으로. 2개 이하면 스키마(AGENTS.md)를 다시 손보거나 자료 큐레이션 기준을 좁히세요.

목표: 한 명에서 작은 팀(3~5명)으로 확장하면서 거버넌스 기초를 세움.

- **5~6주차** – IAM 권한 분리(원본 read-only, Wiki PR 제안만). KMS로 모든 자료 암호화. 팀원 2~3명 추가 초대.
- **7주차** – CloudTrail 활성화. “누가 어떤 자료를 어떻게 바꿨는가”를 추적할 수 있어야 함. Macie 로 PII 자동 탐지 시작.
- **8주차** – 직군별 인터페이스 첫 시도. 개발자는 CLI, 기획자는 Slack 봇 또는 웹 UI. “파일”을 직접 보여주지 않는 사용자층을 정의.

60일 차 게이트 점검:

1. 팀원들이 Wiki에 “질문”을 던지는 횟수가 주당 5회 이상인가?
 2. 동일한 질문이 두 번 이상 나오지 않는가? (즉 Wiki가 정리 효과를 내는가)
 3. 자동 평가 메트릭상 **Drift Rate가 5% 미만**이고 Faithfulness가 유지되는가?
 4. 신규 입사자나 새 팀원에게 Wiki를 “기본 온보딩 자료”로 제시할 수 있는가?
 5. 보안/감사 측면에서 “민감 자료가 어디에 있고 누가 봤는가”에 답할 수 있는가?
- 여기서 “예”가 3개 이상이면 Phase 2(하이브리드)를 검토할 시점입니다.

목표: 일회성 실험에서 “조직의 운영 시스템”으로 전환. Phase 2 도입 결정.

- **9주차** – Phase 2 신호 점검. 다음 중 둘 이상이면 RAG 계층 추가 결정.
 - Wiki만으로 “모르는 영역”에서 답이 부실하다는 피드백 누적
 - 자료가 1,000건을 넘어 Bedrock context에 들어가지 않음
 - “출처를 chunk 단위로 보여달라”는 요구가 생김
 - 빠르게 갱신되는 데이터(가격, 인사 정보 등)가 Wiki에서 자주 stale
- **10~11주차** – (Phase 2 결정 시) OpenSearch Service 또는 Knowledge Bases for Bedrock 추가. 기존 자료를 RAG로도 인덱싱. Wiki 페이지도 RAG에 색인.
- **12주차** – Bedrock Agent에 라우팅 로직 추가. 사실 회수성 질문은 RAG로, 종합성 질문은 Wiki로. 두 결과를 결합한 답변 패턴 검증.

90일 차 최종 게이트:

1. 신규 입사자 온보딩 시간이 도입 전 대비 30% 이상 단축됐는가?
 2. 시니어 직원의 “반복 질문 응대” 시간이 주당 3시간 이상 감소했는가?
 3. 의사결정 추적(“왜 이 결정을 했는가”)에 Wiki가 1차 답변 소스로 쓰이는가?
 4. 월 비용이 처음 산정한 예산 범위 안에 있는가?
- 이 단계까지 왔다면 시스템은 자기 가치를 증명한 것이고, 이제 “실험”이 아니라 “인프라”입니다.

비용 가이드

- **Day 1~30 (Phase 1, 1명):** 월 \$30~\$50 (S3 + Bedrock 호출 위주)
- **Day 31~60 (Phase 1, 5명):** 월 \$100~\$200
- **Day 61~90 (Phase 2, 5~10명):** 월 \$300~\$800 (OpenSearch 추가가 가장 큰 증분)

이 비용은 “소비된 토큰”과 “인덱싱된 데이터”에 비례하므로, 사용량이 적은 주에는 자동으로 더 떨어집니다.

12. 핵심은 도구의 위치가 바뀐다는 것

지금까지 우리는 AI를 **질문하면 답해주는 도구**로 봤습니다. 똑똑한 검색 엔진이나 요약 기계 같은 거죠.

LLM Wiki에서 AI는 **내 자료를 정리하고 색인하는 보조 도구**로 자리잡습니다. 자료가 들어오면 요약과 교차 참조를 만들고, 표면적 모순을 표시하고, 빈 곳에 대한 질문을 제안합니다. 사람은 **무엇을 모을지 정하고, 결과를 검증하고, 의미를 해석하는** 역할에 집중합니다. 즉 LLM은 정리를 위임받고, 판단은 사람이 유지합니다.

1945년에 미국 과학자 바네바 부시(Vannevar Bush)가 **메멕스(Memex)**라는 개념을 제안했습니다. 개인이 자료를 모으고 그 사이에 의미 있는 연결을 만들어두는 기계요. 80년 동안 이 비전이 실현되지 못한 가장 큰 이유 중 하나는 “그 연결을 누가 다 관리하느냐”였습니다. LLM이 그 기계적 작업을 분담합니다. 다만 “의미 있는” 연결인지를 판단하는 것은 여전히 사람의 몫입니다.

AWS는 이 패턴이 개인 실험을 넘어 **조직의 운영 시스템**으로 자리잡을 수 있도록 받쳐주는 역할입니다. 마크 다운 파일 폴더에서 시작해도 좋지만, 그것이 회사의 의사결정에 인용되는 순간 보안, 권한, 감사, 평가 메트릭, 그리고 잘못 정리된 정보를 잡아내는 검증 게이트가 따라와야 하니까요.

요약하면, 더 똑똑한 모델을 기다리는 것만큼이나 **모델이 만든 텍스트를 어떻게 보관하고 검증할지도** 중요했다는 이야기입니다. LLM Wiki는 “기억”이 아니라 “외부 캐시 색인”이고, 그 색인을 안전하게 만드는 일은 클라우드의 자연스러운 쓰임새 중 하나입니다.

한 줄 정리

- LLM Wiki는 RAG의 “대체”가 아니라 **외부 캐시 색인 계층**이다. RAG가 “질문 시점의 검색”을 말한다면, LLM Wiki는 “질문 사이의 텍스트 정리”를 말한다. 다만 이 정리는 손실 압축이며, 원본을 대체하지 않는다.
- Naive RAG와 비교하면 LLM Wiki는 누적성과 가독성에서 강하지만, GraphRAG, Modular RAG, Hybrid RAG는 각자 다른 강점을 가진다. 단일 정답은 없다.
- **Agentic AI와는 경쟁이 아니라 보완 관계**다. 에이전트가 작업을 실행하고, LLM Wiki가 그 작업의 맥락과 결과를 보존한다. 둘을 결합하면 “배우는 에이전트 시스템”에 가까워지지만 hallucination 영속화 위험도 함께 커진다.
- **Hallucination 누적, 측정 메트릭 부재, 고위험 도메인에서의 오용**은 도입 전 반드시 고려할 위험이다. 인간 검토 게이트와 자동 평가(RAGAS류)는 “권장”이 아니라 “필수”다.
- **조직 자료의 외부화는 fragility를 동반한다**. 외부화된 자료는 사람의 머릿속 자료보다 변조와 누적 오류에 취약할 수 있다. Trust level 분류와 정기 검증 없이 외부화하면 오히려 위험이 커진다.
- 실제 운영 환경에서는 **하이브리드**가 답인 경우가 많다. 원본은 RAG로 검색하고, LLM Wiki가 정리한 종합도 다시 RAG에 넣어 양쪽이 맞물리게 한다.
- AWS의 S3, Bedrock, OpenSearch/Kendra, Knowledge Bases, Lambda, CodeCommit 조합은 이 하이브리드 운영을 인프라 부담 없이 시작할 수 있게 해 준다. Bedrock 락인이 우려되면 SageMaker JumpStart self-hosted 옵션과 LiteLLM 추상화로 이식성을 확보할 수 있다.
- 비즈니스 가치의 핵심은 신규 온보딩 시간 단축, 반복 질문 감소, 조직 자료의 외부화이며, 작게 시작해 점진적으로 확장하되 **실패 비용**도 함께 추정하는 것이 가장 합리적인 도입 경로다.